LAB MANUAL



Lab Practice IV(Deep Learning)

B.E

(With effect from- AcademicYear2026- 27)

Department

of

Artificial Intelligence & Machine Learning

Zeal Education Society's

ZEAL COLLEGE OF ENGINEERING & RESEARCH, PUNE-41

(An Autonomous Institute Affiliated to Savitribai Phule Pune University)`

NAACA credited with A+ Grade/ISO21001:2018

VISION AND MISSION

Institute:

Vision	To be a premier institute in technical education by imparting academic excellence, research, social and entrepreneurial attitude.
Mission	1. To achieve academic excellence through innovative teaching and learning process. 2. To imbibe the research culture for addressing industry and societal needs. 3. To inculcate social attitude through community engagement initiatives. 4. To provide conducive environment for building the entrepreneurial skills

Department:

Vision	**To produce competent artificial intelligence and Machine Learning professionals to serve the needs of the society.**
Mission	**M1:** To impart quality, skill-based and value-based education to the students in the field of Artificial Intelligence and Machine Learning **M2:** To identify industrial requirements and enhance the students' expertise. **M3:** To build an ecosystem that will have an ethical impact on the society.

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & MACHINE LEARNING

SYLLABUS

Savitribai Phule Pune Univesity, Pune

Final Year Artificial Intelligence and Machine Learning (2020 Course)

418543: Deep Learning for AI

Teaching Scheme:	Credit Scheme:	Examination Scheme:
Theory (TH) : 3 hrs/week	03 Credits	Mid_Semester : 30 Marks End_Semester : 70 Marks

Prerequisite Courses:
1. Machine Learning 2. Engineering Mathematics

Course Objectives:

1. To introduce the theoretical foundations, algorithms, methodologies, and application of neural networks and deep learning.
2. To design and develop an application-specific deep learning model.
3. To analyze real-world AI applications.

Course Outcomes:

On completion of the course, students will be able to–

CO1. Comprehend the theoretical foundations, algorithms, and methodologies of Deep Learning.
CO2. Apply the concepts of Convolution Neural Networks and use of popular CNN architectures.
CO3. Compare Feed Forward Neural Networks and Recurrent Neural Networks and learn modelling the time dimension using RNN and LSTM.
CO4. Elaborate unsupervised deep learning algorithms like Auto-encoders.
CO5. Explore Representation Learning and Transfer Learning techniques using variants of CNN architecture.
CO6. Evaluate the performance of deep learning algorithms and provide solutions to various real-world applications.

COURSE CONTENTS

Unit I	Fundamentals of Deep Learning	(07 hrs)
What is Deep Learning, Multilayer Perceptron, Feed forward neural, Back propagation, Gradient descent, Vanishing gradient problem, **Activation Functions:** RELU, LRELU, ERELU, Optimization Algorithms, **Hyperparameters:** Layer size, Magnitude (momentum, learning rate), Regularization (dropout, drop connect, L1, L2)		
Mapping of Course Outcomes for Unit I	**CO1**	
Unit II	**Convolutional Neural Network:**	**(07 hrs)**

Introduction to CNN, Convolution Operation, Parameter Sharing, Equivariant Representation, Pooling, Variants of the Basic Convolution Function, The basic architecture of CNN, Popular CNN Architecture – AlexNet.		
Case Study	**WEKA Tool**	
Mapping of Course Outcomes for Unit II	**CO2**	
Unit III	**Recurrent Neural Networks**	**(07 hrs)**
Recurrent Neural Networks: Types of Recurrent Neural Networks, Feed-Forward Neural Networks vs Recurrent Neural Networks, Long Short-Term Memory Networks (LSTM), Encoder Decoder architectures, Recursive Neural Networks BI, Role of DW in BI, BI system components, Components of Data Warehouse Architectures.		
Case Study	**Learn Different ETL Tools**	
Mapping of Course Outcomes for Unit III	**CO3,CO4**	
Unit IV	**Autoencoders**	**(07 hrs)**
Under complete Auto encoders, Regularized Autoencoders-Sparse Autoencoders, Stochastic Encoders and Decoders, Denoising Autoencoders, Contractive Autoencoders, Applications of Autoencoders.		
Mapping of Course Outcomes for Unit IV	**CO4**	
Unit V	**Representation Learning**	**(07 hrs)**
Greedy Layer wise Pre-training, Transfer Learning and Domain Adaption, Distributed Representation, Variants of CNN: DenseNet.		
Case Study	**Graph Mining and Social Network Analysis**	

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & MACHINE LEARNING

1.		
2. Mapping of Course **3. Outcomes for Unit V**	**4. CO5**	
5. Unit VI	6. **Applications of Deep Learning**	**a. (07 I**
7. Generative Adversarial Networks – Generator, Discriminator, Training, GAN variants; Autoenc Architecture. Denoising and Sparcity. 8. Case Study - DALL-E, DALL-E 2 and IMAGEN		
9. Case Study	**10. Mining Spatial, Multimedia, Text and Web Data**	
11. Mapping of Course Outcomes **12. for Unit VI**	**13. CO6**	
14. Text Books:		
15. Josh Patterson, Adam Gibson "Deep Learning: A Practitioner's Approach", O'Reilly Media, 2017 16. Ian Goodfellow, YoshuaBengio and Aaron Courville, "Deep Learning", MIT Press, 2017. 17. Nikhil Buduma, "Fundamentals of Deep Learning Designing Next-Generation Machine Intelligen Algorithms" O'Reilly		
18. **Reference Books:**		
19. Umberto Michelucci "Applied Deep Learning. A Case-based Approach to Understanding. 20. Deep Neural Networks" Apress, 2018. 21. Kevin P. Murphy "Machine Learning: A Probabilistic Perspective", The MIT Press, 2012. 22. Giancarlo Zaccone, Md. Rezaul Karim, Ahmed Menshawy "Deep Learning with TensorFlow: Expl neural networks with Python", Packt Publisher, 2017. 23. Antonio Gulli, Sujit Pal "Deep Learning with Keras", Packt Publishers, 2017. 24. Francois Chollet "Deep Learning with Python", Manning Publications, 2017.		

List of Experiments:

25. Study of Deep Learning Packages: Tensorflow, Keras, Theano and PyTorch. Document the distinct features and functionality of the packages.

26. **Note: Use a suitable dataset for the implementation of the following assignments.**
27. Implementing Feed-forward neural networks with Keras and TensorFlow
    a. Import the necessary packages
    b. Load the training and testing data (MNIST/CIFAR10)
    c. Define the network architecture using Keras
    d. Train the model using SGD
    e. Evaluate the network
    f. Plot the training loss and accuracy

28. Build the Image classification model by dividing the model into the following four stages:
    a. Loading and preprocessing the image data
    b. Defining the model's architecture

c. Training the model
d. Estimating the model's performance

29. Use Autoencoder to implement anomaly detection. Build the model by using the following:
    a. Import required libraries
    b. Upload/access the dataset
    c. The encoder converts it into a latent representation
    d. Decoder networks convert it back to the original input
    e. Compile the models with Optimizer, Loss, and Evaluation Metrics

30. Implement the Continuous Bag of Words (CBOW) Model. Stages can be:
    a. **Data preparation**
    b. Generate training data
    c. Train model
    d. **Output**

31. Object detection using Transfer Learning of CNN architectures

    a. Load in a pre-trained CNN model trained on a large dataset
    b. Freeze parameters (weights) in the model's lower convolutional layers
    c. Add a custom classifier with several layers of trainable parameters to model
    d. Train classifier layers on training data available for the task

32. **e.** Fine-tune hyperparameters and unfreeze more layers as needed

Text Books:

1. Hands-On Deep Learning Algorithms with Python: Master Deep Learning Algorithms with Extensive Math by Implementing Them Using TensorFlow
2. Python Deep Learning, 2nd Edition by Ivan Vasilv, Daniel Slater, GianmarioSpacagna, Peter Roelants, Valentino Zocca
3. Natural Language Processing with Python Quick Start Guide by Mirant Kasliwal

DO's AND DON'Ts

Do's:

- Please follow instructions precisely as instructed by your Course Teacher.
- Always perform the experiment or activity precisely as directed by Course Teacher.
- Before performing experiment, read Lab manual carefully.
- Follow all written and verbal instructions carefully.
- Remove your shoes before you enter the lab.
- Always keep quiet. Be considerate to other lab users.
- Report any problems with the computer to the Course Teacher in charge.
- Shut down computers properly.
- Switch off power supply of devices, tubes& fans whenever not necessary.
- Arrange your own chair properly before leaving the lab.

Don'ts:

- The use of personal audio or video equipment is prohibited in the laboratory.
- Don't use mobile phones during lab hours.
- Do not bring any food or drinks in the lab.
- Do not touch any part of computer with we hand.
- Do not hit the keys on the computer too hard.
- Don't damage, remove or disconnect labels, parts, cables or equipment.
- Do not install or download any software or modify or delete any system files on any lab computers.

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & MACHINE LEARNING

INDEX

Sr No.	Title	Page No.	Conducted Date	Submission Date	Sign
1	Study of Deep Learning Packages: Tensorflow, Keras, Theano and PyTorch. Document the distinct features and functionality of the packages.				
2	Implementing Feed-forward neural networks with Keras and TensorFlow 1. Import the necessary packages 2. Load the training and testing data (MNIST/CIFAR10) 3. Define the network architecture using Keras 4. Train the model using SGD 5. Evaluate the network 6. Plot the training loss and accuracy				
3	Build the Image classification model by dividing the model into the following four stages: 1. Loading and preprocessing the image data 2. Defining the model's architecture 3. Training the model 4. Estimating the model's performance				
4	Use Autoencoder to implement anomaly detection. Build the model by using the following: 1. Import required libraries 2. Upload/access the dataset 3. The encoder converts it into a latent representation Decoder networks convert it back to the original input 4. **e**Compile the models with Optimizer, Loss, and Evaluation Metrics				
5	Implement the Continuous Bag of Words (CBOW) Model. Stages can be: **a. Data preparation** **b.** Generate training data **c.** Train model **d. Output**				
6	Object detection using Transfer Learning of CNN architecture				

	Load in a pre-trained CNN model trained on a large dataset **b.** Freeze parameters (weights) in the model's lower convolutional layers **c.** Add a custom classifier with several layers of trainable parameters to model **d.** Train classifier layers on training data available for the task **e.** Fine-tune hyperparameters and unfreeze more layers as needed				

Zeal Education Society's

ZEAL COLLEGE OF ENGINEERING & RESEARCH, PUNE–41

(An Autonomous Institute Affiliated to Savitribai Phule Pune University)

*NAACA ccredited with **A**+ Grade/ISO21001:2018*

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & MACHINE LEARNING

CERTIFICATE

This is to certified that Mr./Miss._____

of class _____Division _____Roll no. _____has completed Experiments / Practical work / Assignments satisfactorily in the subject of _____during the Semester - _____of the academic year _____.

Prof. Manjiri Chumbhale
Course Faculty

Dr. Dipali Shende
Head of Department

Zeal Education Society's
ZEAL COLLEGE OF ENGINEERING & RESEARCH, PUNE–41
(An Autonomous Institute Affiliated to Savitribai Phule Pune University)
NAACA ccredited with A+ Grade/ISO21001:2018

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & MACHINE LEARNING

Experiment No.:1

Study of Deep Learning Packages:

Date of Performance: ___

Particulars	**Marks**
Regularity (05)	
Performance (15)	
Understanding (Viva)(05)	
Total (25)	
Signature of Course Teacher	

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & MACHINE LEARNING

Expirement:1

Title:

Study of Deep Learning Packages: TensorFlow, Keras, Theano and PyTorch. Document the distinct features and functionalities of the packages.

Objectives:

1. To understand the basic concepts and importance of deep learning packages.
2. To study the features and functionalities of TensorFlow, Keras, Theano and PyTorch.
3. To compare the advantages and limitations of different deep learning packages.

Theory:

Introduction

Deep learning is a branch of machine learning that uses artificial neural networks with multiple layers to learn patterns from large amounts of data. Deep learning packages provide tools to design, train, test and deploy neural network models.

The four important packages studied in this experiment are TensorFlow, Keras, Theano and PyTorch.

A. TensorFlow

TensorFlow is an open-source machine learning framework developed by Google. It is used to build, train and deploy deep learning models.

Features:

- Supports CPU and GPU computation.
- Provides tools for building and training neural networks.
- Supports image processing and natural language processing.
- Supports model deployment on mobile devices, web applications and servers.
- Provides automatic differentiation to calculate gradients.

Applications: Image classification, speech recognition, object detection and predictive analytics.

B. Keras

Keras is a high-level deep learning API that simplifies the process of creating and training neural networks. Modern Keras supports multiple backends, including TensorFlow.

Features:

- Simple and easy-to-understand syntax.
- Supports sequential and functional model building.
- Provides predefined layers and activation functions.
- Makes model training and evaluation easier.
- Suitable for beginners and rapid prototyping.

Applications: Handwritten digit recognition, image classification and neural network development.

C. Theano

Theano was an open-source Python library used for defining, optimizing and evaluating mathematical expressions, particularly those involving multidimensional arrays. It was widely used in early deep learning research.

Features:

- Supports symbolic mathematical expressions.
- Provides automatic differentiation.
- Historically supported CPU and GPU computation.
- Optimizes mathematical calculations.
- Helped researchers develop early deep learning models.

Applications: Early neural network research and mathematical model experimentation.

D. PyTorch

PyTorch is an open-source machine learning framework initially developed by Facebook AI Research. It is widely used for deep learning research and application development.

Features:

- Provides a Python-friendly programming interface.
- Supports dynamic computation graphs.
- Supports CPU and GPU computation.
- Provides automatic differentiation through its autograd system.
- Offers flexible model development and debugging.

Applications: Computer vision, natural language processing, generative AI and deep learning research.

Conclusion:

We have successfully studied and compared the features, functionalities, and applications of TensorFlow, Keras, Theano, and PyTorch.

Zeal Education Society's
ZEAL COLLEGE OF ENGINEERING & RESEARCH, PUNE–41
(An Autonomous Institute Affiliated to Savitribai Phule Pune University)
NAACA ccredited with A+ Grade/ISO21001:2018

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & MACHINE LEARNING

Experiment No.:2

Implementing Feed-forward neural networks with Keras and TensorFlow

Date of Performance: ___

Particulars	**Marks**
Regularity(05)	
Performance(15)	
Understanding(Viva)(05)	
Total (25)	
Signature of Course Teacher	

Experiment Title

Implement a feed-forward neural network using Keras and TensorFlow, train it using Stochastic Gradient Descent (SGD), evaluate its performance, and plot training loss and accuracy using the MNIST dataset.

Objective:

1. To understand the architecture of a feed-forward neural network.
2. To load and preprocess the MNIST dataset.
3. To build and train a neural network using Keras and TensorFlow.
4. To evaluate the model using test data.
5. To plot training loss and accuracy.

Theory

Feed-Forward Neural Network

A feed-forward neural network is an artificial neural network in which information moves in one direction, from the input layer through hidden layers to the output layer. It does not contain feedback connections.

The network consists of three main layers:

- Input Layer: Receives input data, such as images of handwritten digits.
- Hidden Layer: Processes input data using weights, biases and activation functions.
- Output Layer: Produces the predicted class of the input image.

Tensor Flow and Keras

TensorFlow is a deep learning framework used to build and train neural networks. Keras provides a simple interface for defining the network architecture, training the model and evaluating its performance.

MNIST Dataset

The MNIST dataset contains 70,000 grayscale images of handwritten digits from 0 to 9. Each image has a size of 28 × 28 pixels. The dataset contains 60,000 training images and 10,000 testing images.

Stochastic Gradient Descent (SGD)

SGD is an optimization algorithm that updates the weights and biases of a neural network to minimize the loss function during training.

Loss and Accuracy

- Loss: Measures the difference between predicted and actual outputs.
- Accuracy: Measures the percentage of correctly classified images.

Procedure

1. Import TensorFlow and Matplotlib.
2. Load the MNIST training and testing datasets.
3. Normalize the pixel values of the images.
4. Define the feed-forward neural network using Keras.
5. Compile the model using SGD and the appropriate loss function.
6. Train the model using the training dataset.
7. Evaluate the trained model using the testing dataset.
8. Plot the training and validation loss and accuracy.
9. Observe the results and analyze the model's performance.

Conclusion

We have successfully implemented a feed-forward neural network using Keras and TensorFlow, trained it using SGD on the MNIST dataset, evaluated its performance, and plotted the training loss and accuracy.

Zeal Education Society's
ZEAL COLLEGE OF ENGINEERING & RESEARCH, PUNE–41
(An Autonomous Institute Affiliated to Savitribai Phule Pune University)
NAACA ccredited with A+ Grade/ISO21001:2018

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & MACHINE LEARNING

Experiment No.:3

Build the Image classification model by dividing the model into the following four stages

Date of Performance: ___

Particulars	Marks
Regularity(05)	
Performance(15)	
Understanding(Viva)(05)	
Total (25)	
Signature of Course Teacher	

Experiment Title

To build an image classification model by loading and preprocessing image data, defining the model architecture, training the model, and estimating its performance using TensorFlow and Keras.

Objectives

1. To load and preprocess image data for classification.
2. To define a convolutional neural network (CNN) using Keras.
3. To train the image classification model using training data.
4. To evaluate the model's performance using testing data.

Theory

Image classification is a deep learning task in which a model identifies and assigns a class label to an image.

A Convolutional Neural Network (CNN) is commonly used for image classification because it can automatically learn important image features such as edges, shapes and patterns.

The image classification process consists of four stages:

a. **Loading and preprocessing image data:** The dataset is loaded, and image pixel values are normalized to improve training.

b. **Defining the model architecture:** Convolutional, pooling and dense layers are used to extract image features and predict classes.

c**. Training the model**: The model learns patterns from training images by adjusting its weights and minimizing the loss function.

d. **Estimating model performance**: The trained model is evaluated using test images. Accuracy and loss are used to measure its performance.

In this experiment, the CIFAR-10 dataset is used. It contains 60,000 color images of size 32 × 32 pixels belonging to 10 different classes.

Procedure

1. Import TensorFlow and Matplotlib.
2. Load the CIFAR-10 training and testing datasets.
3. Normalize the image pixel values between 0 and 1.
4. Define a CNN architecture using convolutional, pooling and dense layers.
5. Compile the model with an optimizer, loss function and accuracy metric.
6. Train the model using the training dataset.
7. Evaluate the model using the testing dataset.
8. Plot the training and validation accuracy and loss.
9. Observe and record the model's performance.

Conclusion

We have successfully built and trained an image classification model using TensorFlow and Keras. The model was evaluated using test data, and its performance was analyzed using accuracy and loss plots.

Experiment No.:4

Use Autoencoder to implement anomaly detection. Build the model by using the following:

Date of Performance: ___

Particulars	Marks
Regularity(05)	
Performance(15)	
Understanding(Viva)(05)	
Total (25)	
Signature of Course Teacher	

Zeal Education Society's

ZEAL COLLEGE OF ENGINEERING & RESEARCH, PUNE–41

(An Autonomous Institute Affiliated to Savitribai Phule Pune University)

*NAACA ccredited with **A**+ Grade/ISO21001:2018*

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & MACHINE LEARNING

Experiment Title

To implement anomaly detection using an autoencoder by loading a dataset, building encoder and decoder networks, and compiling the model using an optimizer, loss function, and evaluation metrics.

Objectives

1. To understand the concept of autoencoders and anomaly detection.
2. To load and preprocess a dataset for anomaly detection.
3. To build an encoder and decoder using TensorFlow and Keras.
4. To train an autoencoder to reconstruct normal data.
5. To detect anomalies using reconstruction error.

Theory

Anomaly detection is the process of identifying data points that differ significantly from normal patterns. It is used in fraud detection, network security, industrial fault detection, and healthcare monitoring.

An autoencoder is a type of neural network that learns to reproduce its input at the output. It consists of two main components:

- **Encoder:** Compresses the input data into a smaller representation called the latent representation.
- **Decoder:** Reconstructs the original input from the latent representation.

An autoencoder is trained mainly on normal data. It learns to reconstruct normal patterns accurately. When an unusual input is given, it may produce a larger reconstruction error. This error can be used to identify potential anomalies.

The reconstruction error is commonly calculated using Mean Squared Error (MSE).

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(x_i - \hat{x}_i)^2$$

Where:

- (x_i) = original input
- (hat{x}_i) = reconstructed output
- (n) = number of input features

If the reconstruction error exceeds a selected threshold, the data point is classified as a potential anomaly.

Procedure / Steps

1. Import the required libraries, including NumPy, TensorFlow, Matplotlib and Scikit-learn.
2. Create a dataset containing normal and anomalous samples.
3. Separate the dataset into training and testing sets.
4. Use only normal training samples to train the autoencoder.
5. Build an encoder to create a latent representation of the input.
6. Build a decoder to reconstruct the original input.
7. Compile and train the autoencoder using the Adam optimizer and MSE loss.
8. Calculate reconstruction errors for the test samples.
9. Set a threshold based on normal training reconstruction errors and identify potential anomalies.
10. Evaluate the predictions and plot the loss and reconstruction errors.

Conclusion

We have successfully implemented anomaly detection using an autoencoder. The encoder compressed the input data, and the decoder reconstructed it. Anomalies were identified using reconstruction errors and a threshold.

Zeal Education Society's
ZEAL COLLEGE OF ENGINEERING & RESEARCH, PUNE–41
(An Autonomous Institute Affiliated to Savitribai Phule Pune University)
NAACA ccredited with A+ Grade/ISO21001:2018

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & MACHINE LEARNING

Experiment No.:5

Implement the Continuous Bag of Words (CBOW) Model. Stages can be:

Date of Performance: _

Particulars	Marks
Regularity(05)	
Performance(15)	
Understanding(Viva)(05)	
Total (25)	
Signature of Course Teacher	

Experiment Title

To implement the Continuous Bag of Words (CBOW) model to predict a target word based on its surrounding context words.

Objectives

1. To understand the concept of the CBOW model.
2. To prepare text data for word prediction.
3. To generate training data using context words and target words.
4. To train a CBOW model using TensorFlow and Keras.
5. To predict a target word from the given context words.

Theory

The Continuous Bag of Words (CBOW) model is a word embedding technique used in Natural Language Processing (NLP). It predicts a target word using the surrounding context words in a sentence.

For example, consider the sentence:

"Artificial intelligence is changing the world."

If the target word is "changing", the context words may be "intelligence", "is", "the", and "world", depending on the selected context window.

The CBOW model consists of the following stages:

- Data Preparation: Convert text into lowercase and tokenize it into words.
- Generate Training Data: Use surrounding words as input and the target word as output.
- Train Model: Learn word representations and predict the target word.
- Output: Predict the most likely target word from the given context.

CBOW is commonly used in NLP applications such as word embeddings, text analysis, and language modeling.

Procedure / Steps

1. Import NumPy and TensorFlow libraries.
2. Prepare a sample text corpus and convert it into lowercase tokens.
3. Assign a unique integer index to each word.
4. Generate context words and their corresponding target words using a fixed window size.
5. Define the CBOW model using an embedding layer, average pooling layer, and output layer.
6. Compile the model using the Adam optimizer and categorical prediction loss.
7. Train the model using the generated context-target pairs.
8. Provide context words to the trained model and predict the target word.
9. Display the context words, actual target word, and predicted target word.

Conclusion

We have successfully implemented the Continuous Bag of Words (CBOW) model using TensorFlow and Keras. The model was trained using context-target word pairs and used to predict a target word from its surrounding

Zeal Education Society's
ZEAL COLLEGE OF ENGINEERING & RESEARCH, PUNE–41
(An Autonomous Institute Affiliated to Savitribai Phule Pune University)
NAACA ccredited with A+ Grade/ISO21001:2018

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & MACHINE LEARNING

Experiment No.:6

Object detection using Transfer Learning of CNN architectures

Date of Performance: ___

Particulars	Marks
Regularity(05)	
Performance(15)	
Understanding(Viva)(05)	
Total (25)	

Experiment Title

To implement object detection using transfer learning of a pre-trained Convolutional Neural Network (CNN) by loading a pre-trained model, freezing lower convolutional layers, adding a custom classifier, training the model, and fine-tuning its parameters.

Objectives

1. To understand transfer learning and its applications in computer vision.
2. To load a pre-trained CNN model trained on a large dataset.
3. To freeze pre-trained layers and add a custom classification layer.
4. To train the model on task-specific image data.
5. To fine-tune model parameters and evaluate its performance.

Theory

Transfer learning is a deep learning technique in which a model trained on a large dataset is reused for a new, related task. It reduces training time and the amount of data required.

A Convolutional Neural Network (CNN) is commonly used for image-related tasks because it learns features such as edges, shapes, textures, and objects.

The main stages of transfer learning are:

- Load a pre-trained model: Use a CNN such as MobileNetV2, trained on the ImageNet dataset.
- Freeze the base layers: Keep the pre-trained weights unchanged during initial training.
- Add a custom classifier: Add trainable layers to classify images into the required categories.
- Train the classifier: Train the new layers using task-specific image data.
- Fine-tune the model: Unfreeze selected upper layers and train them with a low learning rate to improve performance.

Important: Image classification predicts a class for an entire image, whereas object detection identifies objects and their locations using bounding boxes. The following practical implements transfer learning for image classification. A complete object detection system also requires bounding-box annotations and a detection architecture.

Procedure

1. Import TensorFlow, NumPy and Matplotlib.
2. Load the CIFAR-10 training and testing datasets.
3. Resize the images to the required input dimensions.
4. Load the pre-trained MobileNetV2 model with ImageNet weights.
5. Freeze the pre-trained convolutional layers.
6. Add a custom classifier with pooling, dense and dropout layers.
7. Compile and train the model using the training dataset.
8. Unfreeze selected upper layers and fine-tune using a low learning rate.
9. Evaluate the model using test data and record its accuracy and loss.
10. Plot the training accuracy and analyze the results.

Conclusion

We have successfully implemented transfer learning using a pre-trained MobileNetV2 model. The custom classifier was trained, selected layers were fine-tuned, and the model's performance was evaluated using test data.